

Node.js

Summary Session 10



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

База данных с доступом на чтение

hostname: ich-db.edu.itcareerhub.de

username: ich1

password: password



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Какие основные ранжирующие функции вы помните?

Основные ранжирующие функции

ROW_NUMBER()

RANK()

DENSE_RANK()

NTILE()

Экспресс-опрос

?

Что такое ROW_NUMBER()?



ROW_NUMBER()

Это функция, которая присваивает уникальный номер каждой строке в рамках окна.

Экспресс-опрос

?

Что такое RANK()?



RANK()

Это функция, которая присваивает ранги строкам, пропуская ранги при одинаковых значениях.

Экспресс-опрос

?

Что такое DENSE_RANK()?



DENSE_RANK()

Это функция, которая присваивает ранги строкам, без пропуска рангов при одинаковых значениях.

Экспресс-опрос

?

Что такое NTILE()?



NTILE()

Это функция, которая делит строки на равные части (квантили) и присваивает номер группе.

Применение на практике

Анализ позиций и рангов

Упрощение сложных
запросов

Упорядочение и
группировка данных

Рейтинг товаров или
сотрудников

Замена подзапросов

Группировка данных

Оценка результатов
конкурсов

Избежание дублирования
данных

Создание отчетов

Применение на практике

Анализ позиций и рангов

Упрощение сложных
запросов

Упорядочение и
группировка данных

Рейтинг товаров или
сотрудников

Замена подзапросов

Группировка данных

Оценка результатов
конкурсов

Избежание дублирования
данных

Создание отчетов

Применение на практике

Анализ позиций и рангов

Упрощение сложных
запросов

Упорядочение и
группировка данных

Рейтинг товаров или
сотрудников

Замена подзапросов

Группировка данных

Оценка результатов
конкурсов

Избежание дублирования
данных

Создание отчетов

Ранжирующие функции



Общий синтаксис

```
SELECT column1,
       ranking_function(column2) OVER (
           PARTITION BY column3
           ORDER BY column4
       ) AS rank_column
FROM table_name;
```

Пояснения

- `ranking_function(column2)` - Оконная функция ранжирования, которая применяется.
- `PARTITION BY column3` - Разделяет данные на группы, если требуется.
- `ORDER BY column4` - Устанавливает порядок строк в рамках окна для ранжирования.

Пример 1. ROW_NUMBER()

Исходные данные

SaleID	SaleDate	SaleAmount
1	2024-01-01	100
2	2024-01-02	150
3	2024-01-03	200

Пример 1. ROW_NUMBER()

Решение и результат

```
SELECT SaleID, SaleDate, SaleAmount,
       ROW_NUMBER() OVER (ORDER BY SaleDate) AS RowNum
FROM Sales;
```

SaleID	SaleDate	SaleAmount	RowNum
1	2024-01-01	100	1
2	2024-01-02	150	2
3	2024-01-03	200	3

Пример 2. RANK()

Исходные данные

ProductID	DepartmentID	SaleAmount
1	10	300
2	10	300
3	10	200
4	20	500
5	20	400

Пример 2. RANK()

Решение и результат

```
SELECT ProductID, DepartmentID, SaleAmount,
       RANK() OVER (PARTITION BY DepartmentID ORDER BY SaleAmount DESC) AS Rank
FROM ProductSales;
```

ProductID	DepartmentID	SaleAmount	Rank
1	10	300	1
2	10	300	1
3	10	200	3
4	20	500	1
5	20	400	2

Пример 3. DENSE_RANK()

Исходные данные

ProductID	SaleAmount
1	300
2	300
3	200
4	500
5	400

Пример 3. DENSE_RANK()

Решение и результат

```
SELECT ProductID, SaleAmount,
       DENSE_RANK() OVER (ORDER BY SaleAmount DESC) AS DenseRank
FROM ProductSales;
```

ProductID	SaleAmount	DenseRank
1	300	1
2	300	1
3	200	3
4	500	1
5	400	2

Пример 4. NTILE()

Исходные данные

SaleID	SaleAmount
1	100
2	200
3	300
4	400
5	500

Пример 4. NTILE()

Решение и результат

```
SELECT SaleID, SaleAmount,
       NTILE(3) OVER (ORDER BY SaleAmount) AS Quartile
FROM Sales;
```

SaleID	SaleAmount	Quartile
1	100	1
2	200	1
3	300	2
4	400	2
5	500	3

Экспресс-опрос

?

Что такое оконные функции смещения и выбора?



Оконные функции смещения и выбора

Эти функции позволяют выполнять вычисления, которые зависят от значений в других строках в пределах определенного окна.

Экспресс-опрос

?

Какие основные оконные функции смещения и выбора вы помните?

Основные агрегирующие оконные функции

LEAD()

LAG()

FIRST_VALUE()

LAST_VALUE()

NTH_VALUE()

Оконные функции смещения и выбора



Общий синтаксис

```
SELECT column1,  
       window_function(column2) OVER (  
           PARTITION BY column3  
           ORDER BY column4 ) AS  
result_column  
FROM table_name;
```

Пояснение

- `window_function(column2)`: Оконная функция смещения или выбора, которая применяется
- `PARTITION BY column3`: Разделяет данные на группы, если требуется
- `ORDER BY column4`: Устанавливает порядок строк в рамках окна

Экспресс-опрос

?

Что такое LEAD()?



LEAD()

Это функция, которая возвращает значение из следующей строки в пределах окна, которое определяется ORDER BY.

Пример использования функции LEAD()

Исходные данные

1. Найти значение SaleAmount следующей продажи по дате.

OrderID	CustomerID	OrderDate
1	101	2024-01-01
2	101	2024-01-05
3	102	2024-01-02
4	101	2024-01-10
5	102	2024-01-07

Пример использования функции LEAD()

Решение и результат

```
SELECT SaleID, SaleDate, SaleAmount,
       LEAD(SaleAmount, 1) OVER (ORDER BY SaleDate) AS NextSaleAmount
FROM Sales;
```

OrderID	CustomerID	OrderDate	OrderAmount
1	101	2024-01-01	300
2	101	2024-01-05	500
4	101	2024-01-10	700
3	102	2024-01-02	150
5	102	2024-01-07	200

Экспресс-опрос

?

Что такое LAG()?



LAG()

Это функция, которая возвращает значение из предыдущей строки в пределах окна.

Пример использования функции LAG()

Исходные данные

1. Найти значение SaleAmount предыдущей продажи по дате.

SaleID	SaleDate	SaleAmount
1	2024-01-01	100
2	2024-01-02	150
3	2024-01-03	200

Пример использования функции LAG()

Решение и результат

```
SELECT SaleID, SaleDate, SaleAmount,
       LAG(SaleAmount, 1) OVER (ORDER BY SaleDate) AS PrevSaleAmount
FROM Sales;
```

SaleID	SaleDate	SaleAmount	PrevSaleAmount
1	2024-01-01	100	NULL
2	2024-01-02	150	100
3	2024-01-03	200	150

Экспресс-опрос

?

Что такое FIRST_VALUE()?



FIRST_VALUE()

Это функция, которая возвращает первое значение в рамках окна.

Пример использования функции FIRST_VALUE()

Исходные данные

1. Найти первую цену продажи для каждого месяца.

OrderID	OrderDate	UnitPrice
1	2024-01-01	10
2	2024-01-05	20
3	2024-02-01	15
4	2024-02-15	25

Пример использования функции FIRST_VALUE()

Решение и результат

```
SELECT OrderID, OrderDate, UnitPrice,
       FIRST_VALUE(UnitPrice) OVER (PARTITION BY EXTRACT(YEAR FROM OrderDate), EXTRACT(MONTH
FROM OrderDate) ORDER BY OrderDate) AS FirstPriceOfMonth
FROM OrderDetails;
```

OrderID	OrderDate	UnitPrice	FirstPriceOfMonth
1	2024-01-01	10	10
2	2024-01-05	20	10
3	2024-02-01	15	15
4	2024-02-15	25	15

Экспресс-опрос

?

Что такое LAST_VALUE()?



LAST_VALUE()

Это функция, которая возвращает последнее значение в рамках окна.

Пример использования функции LAST_VALUE()

Исходные данные

1. Найти последнюю цену продажи для каждого месяца.

OrderID	OrderDate	UnitPrice
1	2024-01-01	10
2	2024-01-05	20
3	2024-02-01	15
4	2024-02-15	25

Пример использования функции LAST_VALUE()

Решение и результат

```
SELECT OrderID, OrderDate, UnitPrice,
       LAST_VALUE(UnitPrice) OVER (PARTITION BY EXTRACT(YEAR FROM OrderDate), EXTRACT(MONTH
FROM OrderDate) ORDER BY OrderDate) AS LastPriceOfMonth
FROM OrderDetails;
```

OrderID	OrderDate	UnitPrice	LastPriceOfMonth
1	2024-01-01	10	20
2	2024-01-05	20	20
3	2024-02-01	15	25
4	2024-02-15	25	25

Экспресс-опрос

?

Что такое NTH_VALUE()?



NTH_VALUE()

Это функция, которая возвращает Возвращает
n-е значение в рамках окна.

Пример использования функции NTH_VALUE()

Исходные данные

1. Найти третью цену продажи для каждого месяца.

OrderID	OrderDate	UnitPrice
1	2024-01-01	10
2	2024-01-05	20
3	2024-01-10	30
4	2024-02-01	15
5	2024-02-15	25
6	2024-02-20	35

Пример использования функции NTH_VALUE()

Решение и результат

```
SELECT OrderID, OrderDate, UnitPrice,
       NTH_VALUE(UnitPrice, 3) OVER (PARTITION BY EXTRACT(YEAR FROM OrderDate),
EXTRACT(MONTH FROM OrderDate) ORDER BY OrderDate) AS ThirdPriceOfMonth
FROM OrderDetails;
```

OrderID	OrderDate	UnitPrice	ThirdPriceOfMonth
1	2024-01-01	10	30
2	2024-01-05	20	30
3	2024-01-10	30	30
4	2024-02-01	15	35
5	2024-02-15	25	35
6	2024-02-20	35	35



ВОПРОСЫ





ЗАДАНИЕ



Задание

Завершить работу над задачами с практикумов.



Задание 1

Таблица `purchase_order_details`

1. Для каждого `product_id` выведите дату его получения `date_received`, предыдущую и последующую даты получения этого продукта. Оставьте только строки где `date_received` не является пропуском.
2. Найдите время отправки заказа `date_received` и время отправки предыдущего заказа. Для начала оставьте только уникальные пары `purchase_order_id`, `date_received` и отфильтруйте строки, там где `date_received` не указано. Запишите результат в CTE и дальше работайте с ним.
3. Выведите максимальное количество `quantity` и минимальный `unit_cost` для каждого `inventory_id` с помощью функции `FIRST_VALUE`.
4. Выведите одно значения – насколько в среднем отличается `unit_cost` для каждой строки от максимального `unit_cost`.
5. Выберите ТОП 5 продуктов с максимальным `quantity`, используя `DENSE_RANK`.
6. Пронумеруйте строки в соответствии с убывание `inventory_id`. Выведите только 13 строчку.



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

Таблица `purchase_orders`

1. Для каждого `product_id` выведите `inventory_id` а также предыдущий и последующей `inventory_id` по убыванию `quantity`.
2. Выведите максимальный и минимальный `unit_price` для каждого `order_id` с помощью функции `FIRST VALUE`. Выведите `order_id` и полученные значения.
3. Выведите `order_id` и столбец с разнице между `unit_price` для каждой заказа и минимальным `unit_price` в рамках одного заказа.
! Задачу решить двумя способами - с помощью `First Value` и `MIN`.
4. Присвойте ранг каждой строке, используя `RANK` по убыванию `quantity`.
5. Из предыдущего запроса выберите только строки с рангом до 10 включительно.



ВОПРОСЫ



Заключение

